



GUIDE DE CONCEPTION ET D'IMPLÉMENTATION

Système Complet de Vision Artificielle sur SoC FPGA

Plateforme : Terasic DE0-Nano-SoC (Cyclone V SE)

Architecture Hybride : FPGA (VHDL) + HPS (Linux/Qt)

Équipe Projet :

- Haddouche Ziyad : Conception Matérielle (VHDL/Nios/VGA)
- Belmeliani Elias : Interface Système (FPGA/DMA/Ponts)
- Gandon Elias : Applicatif Logiciel (HPS/Qt/Traitement)

Introduction et Contexte du Projet

Dans le cadre de notre projet de système embarqué, l'objectif était de concevoir et de réaliser une architecture complète de vision artificielle sur une plateforme SoC (*System on Chip*).

Nous avons travaillé sur la carte de développement **DE0-Nano-SoC** d'Altera (Terasic), qui présente la particularité d'intégrer une architecture hybride Cyclone V : elle combine une partie logique programmable (FPGA) et un processeur double cœur ARM Cortex-A9 (HPS - *Hard Processor System*).

Cette dualité architecturale nous a permis de tirer parti du meilleur des deux mondes : la parallélisation massive du FPGA pour l'acquisition vidéo rapide, et la souplesse du processeur ARM pour la gestion du système, la connectivité réseau et le traitement d'image applicatif.

1.1. Répartition des tâches au sein du trinôme

La complexité du système a nécessité une division stricte des responsabilités entre le monde matériel (Hardware) et le monde logiciel (Software), tout en assurant une interface de communication robuste entre les deux.

- **Partie Matérielle (Binôme FPGA) :** Mes collègues se sont concentrés sur la conception de l'architecture VHDL. Leurs tâches comprenaient l'interfaçage direct avec le module caméra (D8M), la gestion des signaux vidéo bruts et l'écriture des données en mémoire SDRAM via un contrôleur

DMA (Direct Memory Access). Leur rôle s'arrêtait à la mise à disposition de l'image brute en mémoire.

- **Partie Logicielle et Système (Mon rôle - HPS) :** Ma mission consistait à rendre ce système exploitable et interactif. J'ai pris en charge l'intégralité de la couche logicielle s'exécutant sur le processeur ARM (HPS). Mes responsabilités se sont articulées autour de trois axes principaux :
 - **L'administration système :** L'installation et la configuration d'un système d'exploitation Linux (Debian) pour transformer la carte en un véritable ordinateur embarqué connecté.
 - **Le contrôle matériel :** La configuration des périphériques (notamment le capteur caméra via le bus I2C) et la gestion de la communication avec le FPGA.
 - **L'application de traitement :** Le développement d'une interface graphique en C++ (Qt) capable de récupérer le flux vidéo depuis la mémoire partagée et d'appliquer des algorithmes de vision par ordinateur (détection de contours) en temps réel.

Ce rapport détaille les étapes techniques de cette mise en œuvre complète.

Chapitre 1

Conception Matérielle : Acquisition et Affichage VGA

1.1 1. Vue d'ensemble du système

Ce projet vise à capturer un flux vidéo en temps réel depuis une caméra OV7670, à le stocker dans une mémoire tampon (Frame Buffer) et à l'afficher sur un écran VGA. Le système est piloté par un processeur Nios II (Soft-core) qui gère la configuration de la caméra via le protocole I2C.

Architecture Globale : Le système est divisé en deux parties principales :

- **Hardware (VHDL) :** Gère le flux de données à haute vitesse (Pixels $\rightarrow$ RAM $\rightarrow$ VGA).
- **Software (C / Nios II) :** Gère la configuration bas niveau et l'initialisation de la caméra.

1.2 2. Partie Matérielle (VHDL)

Le code VHDL est l'épine dorsale du projet. Il synchronise trois domaines d'horloge différents :

- `clk50` : Horloge système et lecture RAM.
- `pclk` : Horloge pixel fournie par la caméra (écriture RAM).
- `system_clk_25` : Horloge générée pour le contrôleur VGA.

1.2.1 Code

VHDL

Complet

Voici l'intégralité du code VHDL utilisé pour l'entité Top-Level :

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_arith.all;
4 use ieee.std_logic_unsigned.all;
5
6 entity top is port (
7     clk50, rst_n : in std_logic;
8     sw : in std_logic_vector(3 downto 0);
9     led : out std_logic_vector(7 downto 0);
10
11     -- Interface Camera OV7670
12     xclk : out std_logic;           -- Horloge maitre vers la camera
13     (25MHz)
14     i2c_sda : inout std_logic;     -- Ligne de donnees I2C
15     i2c_scl : inout std_logic;     -- Ligne d'horloge I2C
16     vsync, href, pclk : in std_logic; -- Synchro Verticale, Horizontale
17     et Pixel Clock
18     data_pix : in std_logic_vector(7 downto 0); -- Bus de donnees
19     parallele (8 bits)
20
21     -- Interface VGA
22     hs, vs : out std_logic;         -- Synchro Horizontale et
23     Verticale ecran
24     r, g, b : out std_logic_vector(3 downto 0); -- Canaux couleurs
25
26     -- Signaux de Debug (Visualisation a l'oscillo)
27     pclk_tap, href_tap, vsync_tap : out std_logic
28 );
29 end entity;
30
31 architecture behavior of top is
32
33     -- Signaux internes pour le Nios II et l'I2C
34     signal i2c_sda_in , i2c_scl_in : std_logic;
35     signal i2c_sda_oe , i2c_scl_oe : std_logic;
36     signal system_clk_25 : std_logic := '0';
37
38     -- Signaux de gestion de la RAM (Frame Buffer)
39     signal ram_addr_wr : std_logic_vector(16 downto 0) := (others =>
40     '0');
41     signal ram_data_in : std_logic_vector(11 downto 0) := (others =>
42     '0');
43     signal ram_wren      : std_logic := '0';
44     signal ram_q_out     : std_logic_vector(11 downto 0);
45     signal ram_addr_rd : std_logic_vector(16 downto 0) := (others =>
46     '0');
47     signal write_toggle : std_logic := '0'; -- Bascule pour gerer les 2
48     octets par pixel
49
50     -- Declaration du composant RAM (IP generee par Quartus)
51     component frame_buffer
52     PORT (
53         clock      : IN STD_LOGIC;
54         data       : IN STD_LOGIC_VECTOR (11 DOWNT0 0);

```

```

47      rdaddress      : IN STD_LOGIC_VECTOR (16 DOWNT0 0);
48      wraddress      : IN STD_LOGIC_VECTOR (16 DOWNT0 0);
49      wren            : IN STD_LOGIC      := '0';
50      q               : OUT STD_LOGIC_VECTOR (11 DOWNT0 0)
51  );
52  end component;
53
54  -- Declaration du systeme Qsys (Nios II)
55  component h is
56      port (
57          clk_clk      : in  std_logic := 'X';
58          i2c_0_sda_in  : in  std_logic := 'X';
59          i2c_0_scl_in  : in  std_logic := 'X';
60          i2c_0_sda_oe  : out std_logic;
61          i2c_0_scl_oe  : out std_logic;
62          led_export    : out std_logic_vector(7 downto 0);
63          reset_reset_n : in  std_logic := 'X';
64          sw_export     : in  std_logic_vector(3 downto 0) := (others
=> 'X')
65      );
66  end component h;
67
68  begin
69
70      -- Assignment des signaux de debug
71      href_tap <= href;
72      vsync_tap <= vsync;
73      pclk_tap <= pclk;
74
75      -- Gestion du buffer Tri-state pour l'I2C
76      i2c_scl <= '0' when i2c_scl_oe = '1' else 'Z';
77      i2c_sda <= '0' when i2c_sda_oe = '1' else 'Z';
78      i2c_scl_in <= i2c_scl;
79      i2c_sda_in <= i2c_sda;
80
81      -- Instanciation du processeur Nios II
82      u0 : component h
83          port map (
84              clk_clk      => clk50,
85              i2c_0_sda_in  => i2c_sda_in,
86              i2c_0_scl_in  => i2c_scl_in,
87              i2c_0_sda_oe  => i2c_sda_oe,
88              i2c_0_scl_oe  => i2c_scl_oe,
89              led_export    => led,
90              reset_reset_n => rst_n,
91              sw_export     => sw
92          );
93
94      -- Generation de l'horloge 25MHz (Diviseur par 2)
95      process(clk50)
96      begin
97          if rising_edge(clk50) then
98              system_clk_25 <= not system_clk_25;
99          end if;
100      end process;
101      xclk <= system_clk_25;
102
103      -- Instanciation de la RAM Double Port

```

```

104  u_ram : component frame_buffer
105  port map (
106      clock      => clk50,          -- La RAM tourne a 50MHz
107      wraddress  => ram_addr_wr,
108      data       => ram_data_in,
109      wren       => ram_wren,
110      rdaddress  => ram_addr_rd,
111      q          => ram_q_out
112  );
113
114  -- PROCESS 1 : Capture Video (Clock Domain : PCLK)
115  process(pclk)
116  begin
117      if rising_edge(pclk) then
118          if vsync = '1' then
119              -- Nouvelle image : on reset l'adresse d'ecriture
120              ram_addr_wr <= (others => '0');
121              write_toggle <= '0';
122              ram_wren <= '0';
123          elsif href = '1' then
124              -- Ligne valide
125              write_toggle <= not write_toggle;
126              if write_toggle = '0' then
127                  -- 1er octet (R + G_haut)
128                  ram_data_in(11 downto 8) <= data_pix(7 downto 4); --
129                  ram_data_in(7 downto 4) <= data_pix(2 downto 0) &
130                  '0'; -- Vert (partie haute)
131                  ram_wren <= '0';
132              else
133                  -- 2eme octet (G_bas + B) + Ecriture
134                  ram_data_in(7 downto 4) <= ram_data_in(7 downto 4)
135                  or ('0' & data_pix(7 downto 5));
136                  ram_data_in(3 downto 0) <= data_pix(4 downto 1); --
137                  ram_wren <= '1'; -- Validation de l'ecriture
138
139                  -- Increment adresse (max 320*240 = 76800)
140                  if ram_addr_wr < 76800 then
141                      ram_addr_wr <= ram_addr_wr + 1;
142                  end if;
143              end if;
144          else
145              ram_wren <= '0';
146          end if;
147      end if;
148  end process;
149
150  -- PROCESS 2 : Controleur VGA (Clock Domain : 25MHz)
151  -- Lit la RAM et genere les signaux de synchro
152  process(clk50)
153      variable h_cnt : integer range 0 to 800 := 0;
154      variable v_cnt : integer range 0 to 525 := 0;
155      variable x_pix, y_pix : integer;
156  begin
157      if rising_edge(clk50) then
158          if system_clk_25 = '1' then -- Enable a 25MHz

```

```

158      -- Compteurs H et V standards pour 640x480 @ 60Hz
159      if h_cnt = 799 then
160          h_cnt := 0;
161          if v_cnt = 524 then v_cnt := 0; else v_cnt := v_cnt
+ 1; end if;
162      else
163          h_cnt := h_cnt + 1;
164      end if;
165
166      -- Generation des Sync Pulses
167      if h_cnt < 96 then hs <= '0'; else hs <= '1'; end if;
168      if v_cnt < 2 then vs <= '0'; else vs <= '1'; end if;
169
170      -- Zone d'affichage active
171      if (h_cnt >= 144) and (h_cnt < 784) and (v_cnt >= 35)
and (v_cnt < 515) then
172          x_pix := h_cnt - 144;
173          y_pix := v_cnt - 35;
174
175          -- ZOOM x2 : On divise les coordonnees par 2 pour
lire l'adresse RAM
176          ram_addr_rd <= conv_std_logic_vector((y_pix/2) * 320
+ (x_pix/2), 17);
177
178          r <= ram_q_out(11 downto 8);
179          g <= ram_q_out(7 downto 4);
180          b <= ram_q_out(3 downto 0);
181      else
182          -- Porches (Hors zone active) : Noir
183          r <= (others => '0'); g <= (others => '0'); b <= (
others => '0');
184      end if;
185  end if;
186  end if;
187  end process;
188
189  end behavior;

```

Listing 1.1 – Top-Level VHDL : VGA et Nios

1.2.2 Analyse Technique du VHDL

- **Compression RGB444** : L'OV7670 envoie du RGB565 (16 bits). Pour économiser la mémoire du FPGA (limitée), le code ne garde que les bits de poids fort : 4 pour R, 4 pour G, 4 pour B.
- **Zoom Numérique** : L'image stockée est en QVGA (320x240). L'écran est en VGA (640x480). L'opération $(x_pix/2)$ permet de doubler la taille de chaque pixel à l'affichage.

1.3 3. Partie Logicielle (C sur Eclipse)

Le logiciel configure les registres internes de la caméra via le bus I2C.

GUIDE PRATIQUE : Utilisation de l'IDE Eclipse pour Nios II

Pour programmer le Nios II, nous utilisons l'environnement Eclipse for Nios II :

1. Dans Quartus, aller dans **Tools > Nios II Software Build Tools for Eclipse**.
2. Créer un nouveau projet : **File > New > Nios II Application and BSP from Template**.
3. Sélectionner le fichier .sopcinfo généré par Qsys.
4. Choisir le modèle "Hello World" ou "Blank Project".
5. Copier le code C ci-dessous dans le fichier .c principal.
6. Faire un clic droit sur le projet > **Build Project**.
7. Pour exécuter : Clic droit > **Run As > Nios II Hardware**.

```

1 #include "system.h"
2 #include "altera_avalon_i2c.h"
3 #include "unistd.h"
4 #include "stdio.h"
5
6 #define cam_i2c_addr 0x21
7
8 ALT_AVALON_I2C_DEV_t *i2c_dev;
9 struct regval {uint8_t reg; uint8_t val;};
10
11 static const struct regval ov7670_init[] = {
12     {0x12,0x80}, // COM7 : Reset total
13     {0x11,0x01}, // CLKRC
14     {0x12,0x14}, // COM7 : QVGA + RGB
15     {0x40,0xD0}, // COM15 : RGB565
16     {0x3A,0x04}, // TSLB
17     {0xFF,0xFF},
18 };
19
20 void config_ov7670(void)

```

```

21 {
22     uint8_t buffer[2];
23     printf("Ecriture de la configuration...\n");
24     uint8_t i=0;
25     while (ov7670_init[i].reg != 0xFF)
26     {
27         buffer[0]= ov7670_init[i].reg;
28         buffer[1]= ov7670_init[i].val;
29         alt_avalon_i2c_master_tx(i2c_dev, buffer, 2, 0);
30         usleep(2000);
31         i++;
32     }
33     printf("Configuration terminee.\n");
34 }
35
36 void i2c_init(void)
37 {
38     i2c_dev = alt_avalon_i2c_open("/dev/i2c_0");
39     if (i2c_dev == NULL) {
40         printf("ERREUR : Impossible d'ouvrir /dev/i2c_0\n");
41         exit(1);
42     }
43     alt_avalon_i2c_master_target_set(i2c_dev, cam_i2c_addr);
44 }

```

Listing 1.2 – Code C Nios II pour configuration I2C

1.4 4. Résolution des Problèmes Rencontrés

A. Problème de Communication I2C (Lecture 0x00)

- **Cause** : Pull-Up manquants ou Caméra en Reset.
- **Solution** : Activer "Weak Pull-Up Resistor" sur les pins I2C dans Quartus et connecter RESET au 3.3V.

B. Saturation Mémoire FPGA (Error 170040)

- **Cause** : Le FPGA manque de blocs mémoires M9K pour contenir à la fois le Frame Buffer vidéo et la mémoire de programme du Nios II.
- **Solution** : Dans Platform Designer, réduire la taille du composant "On-Chip Memory" du Nios II de 96 Ko à 40 Ko.

Transition vers l'Architecture Hybride

Bien que l'affichage VGA via le Nios II valide le fonctionnement du capteur et l'acquisition bas niveau, cette architecture reste limitée pour du traitement d'image avancé. Le processeur Softcore sature vite et la mémoire interne du FPGA est trop petite pour stocker plusieurs images haute résolution. Pour atteindre les objectifs du projet (traitement Sobel sur Linux), nous devons passer à une architecture plus performante : le transfert DMA direct vers la SDRAM du processeur HPS. C'est l'objet de la partie suivante.

Chapitre 2

Interface Système : Pont FPGA vers SDRAM HPS

2.1 Introduction et Ressources

L'objectif est de créer une chaîne d'acquisition vidéo matérielle (FPGA) qui écrit directement dans la mémoire du processeur (HPS).

2.1.1 Récupération des ressources (System CD)

Nous utilisons le CD-ROM fourni par Terasic pour récupérer les designs de référence.

1. Aller sur : <https://download.terasic.com/downloads/cd-rom/de0-nano-soc/>
2. Télécharger : **DE0-Nano-SoC_v.1.1.0_HWrevB0_revC0_System**

2.2 Création des IPs (IP Catalog)

Nous adaptons les horloges entre la caméra (25 MHz) et le FPGA (50 MHz).

GUIDE PRATIQUE : Création d'un composant IP dans Quartus

Pour créer la PLL et la FIFO :

1. Ouvrir le panneau **IP Catalog** sur la droite.
2. Chercher le nom du composant (ex : "ALTPLL").
3. Double-cliquer pour lancer le **MegaWizard**.
4. Configurer les paramètres comme indiqué ci-dessous.
5. Cliquer sur **Finish** pour générer le fichier VHDL.

2.2.1 Configuration de la PLL
— **ALTPLL** : Entrée 50 MHz → Sortie 25 MHz.

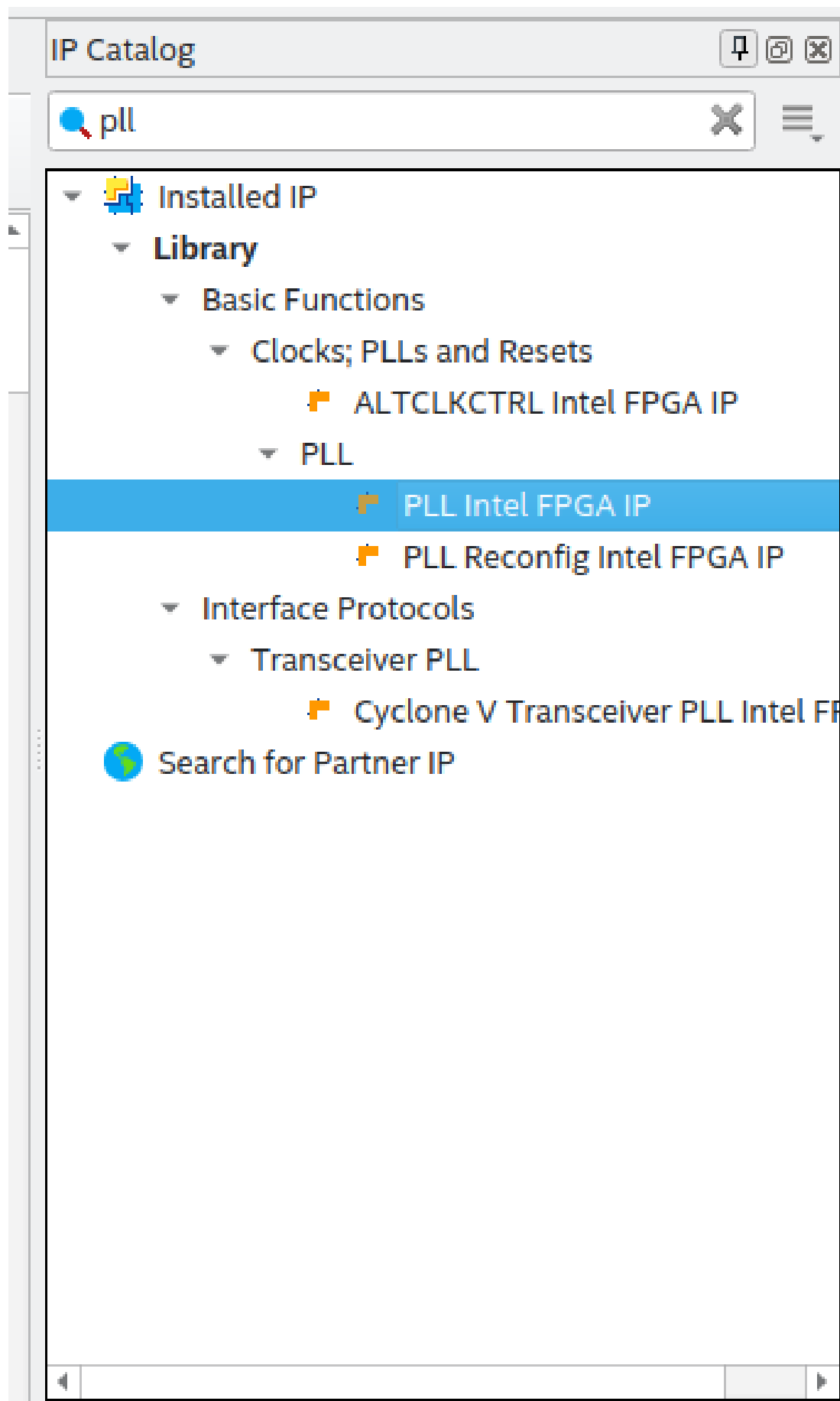


FIGURE 2.1 – PLL : Configuration Entrée
Guide de Conception : DE0-Nano-SoC

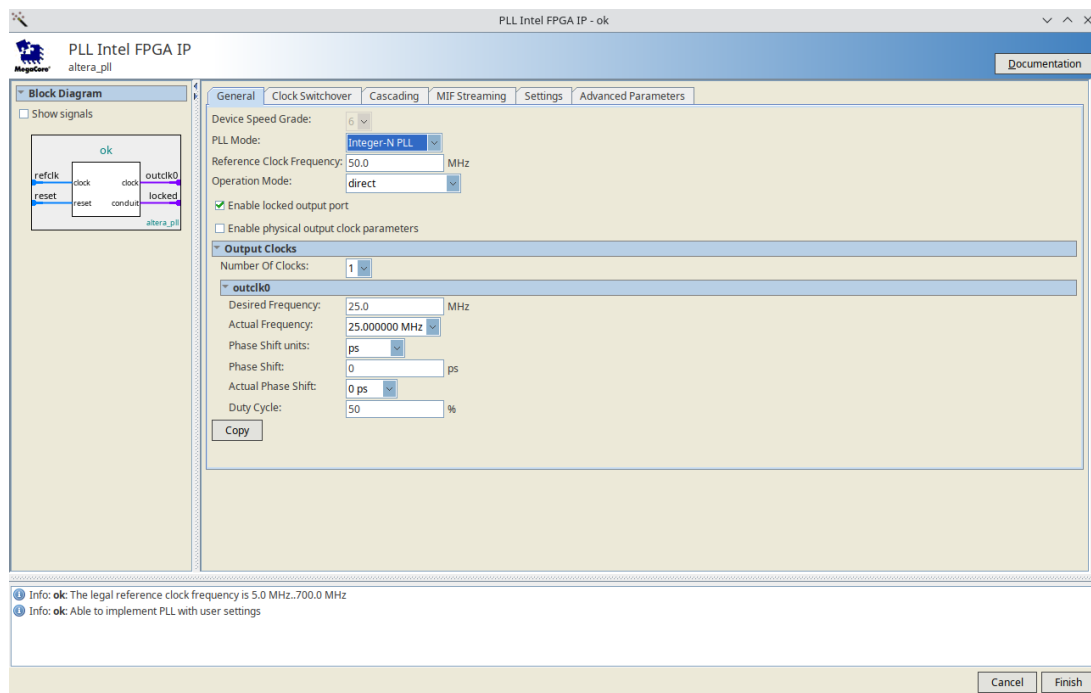


FIGURE 2.2 – PLL : Configuration Sortie

2.2.2 Configuration de la FIFO

Buffer pour traverser les domaines d'horloge.

— FIFO : 16 bits, Mode Dual Clock.

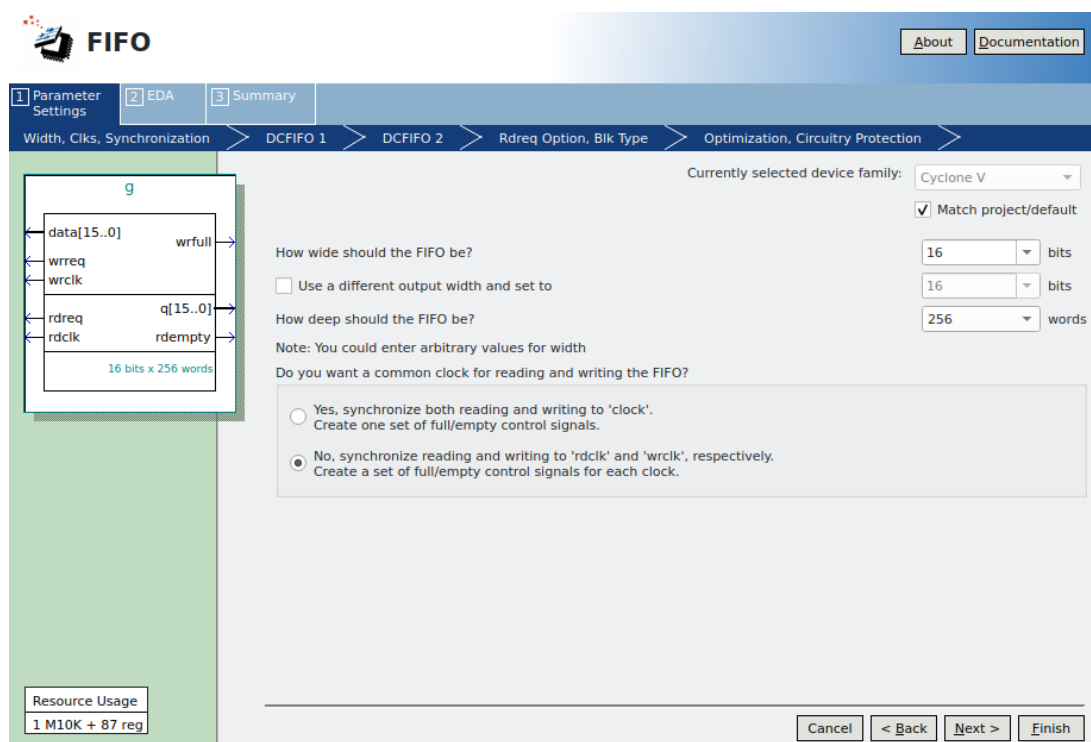


FIGURE 2.3 – Configuration FIFO Dual-Clock

2.3 Le Code VHDL Unifié (Contrôleur DMA)

Ce contrôleur capture les pixels et les écrit via Avalon Master à l'adresse **0x30000000**.

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity fpga_dma_rgb32_simple is
6     port (
7         clk, reset_n      : in  std_logic;
8         -- Avalon Master (Vers HPS)
9         avm_address       : out std_logic_vector(31 downto 0);
10        avm_write          : out std_logic;
11        avm_writedata      : out std_logic_vector(31 downto 0);
12        avm_byteenable     : out std_logic_vector(3 downto 0);
13        avm_waitrequest    : in  std_logic;
14        -- Camera
15        cam_pclk, cam_href, cam_vsync : in std_logic;
16        cam_data           : in  std_logic_vector(7 downto 0);
17        cam_xclk           : out std_logic
18    );
19 end entity;
20
21 architecture rtl of fpga_dma_rgb32_simple is
22     component pll is port (refclk, rst: in std_logic; outclk_0: out
23         std_logic); end component;
24     component camera_fifo is port (...); end component;
25
26     constant BASE_ADDR : unsigned(31 downto 0) := x"30000000";
27     signal dma_addr : unsigned(31 downto 0) := BASE_ADDR;
28 begin
29     -- 1. Horloge Camera (25MHz)
30     u_pll : component pll port map (refclk => clk, rst => not reset_n,
31         outclk_0 => cam_xclk);
32
33     -- 2. Buffer FIFO et Logique DMA
34     process(clk) begin
35         if rising_edge(clk) then
36             if writing = '0' and empty = '0' then
37                 rdreq <= '1'; writing <= '1';
38                 dma_addr <= dma_addr + 4; -- +4 octets (RGB32)
39             elsif waitrequest = '0' then
40                 writing <= '0';
41             end if;
42         end if;
43     end process;
44
45     avm_byteenable <= "1111"; -- Ecriture 32 bits
46     avm_address <= std_logic_vector(dma_addr);
47 end architecture;

```

Listing 2.1 – Code VHDL Maître : fpga_dma_rgb32_simple.vhd

2.4 Intégration Qsys (Platform Designer)

GUIDE PRATIQUE : Intégration d'un composant VHDL dans Qsys

1. Ouvrir **Platform Designer (Qsys)**.
2. Dans l'onglet **IP Catalog**, double-cliquer sur **New Component**.
3. Dans l'onglet **Files**, ajouter votre fichier VHDL et cliquer sur **Analyze Synthesis Files**.
4. Dans l'onglet **Signals**, configurer les interfaces (Avalon Master, Clock, Conduit).
5. Cliquer sur **Finish**. Le composant apparaît dans la liste.
6. L'ajouter au système et relier le port Avalon Master au port `f2h_sdram0_data` du HPS.
7. Cliquer sur **Generate HDL...** et attendre la fin de la compilation (environ 5 min).

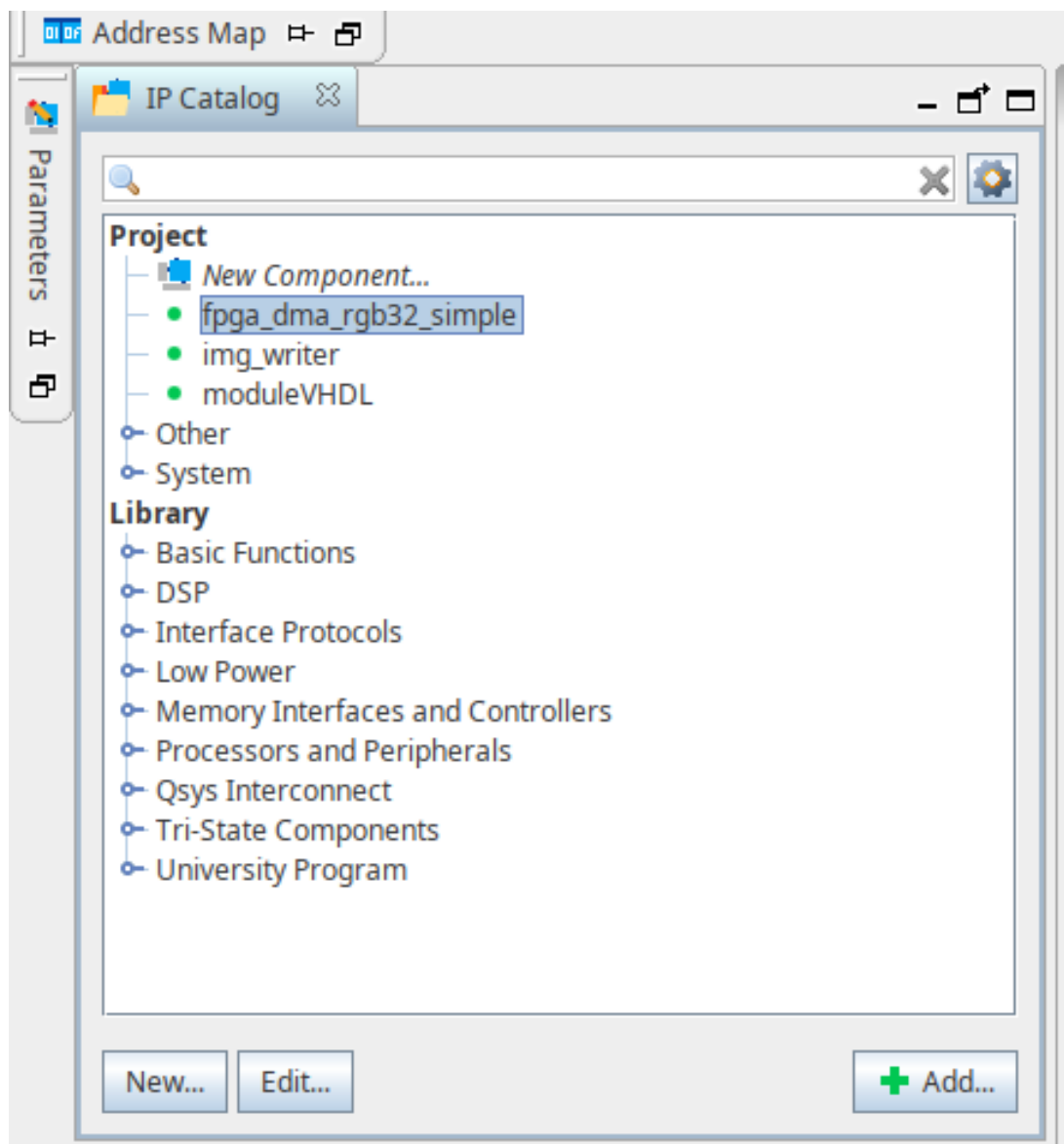


FIGURE 2.4 – Création du composant Qsys

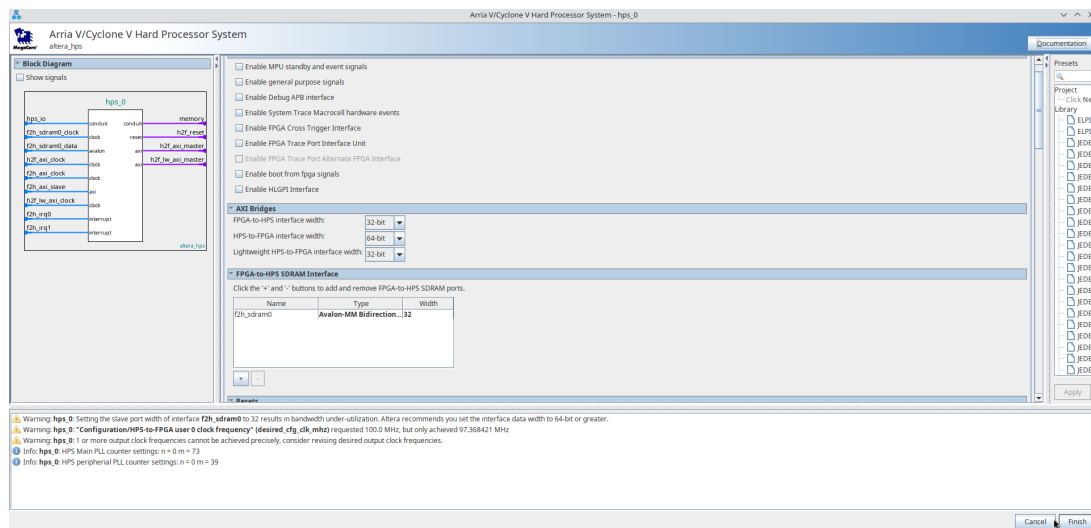


FIGURE 2.5 – Connexion dans le système global

2.5 Schéma Bloc (.BDF) et Pin Planner

2.5.1 Schéma BDF

Connexion graphique du système Qsys aux ports.

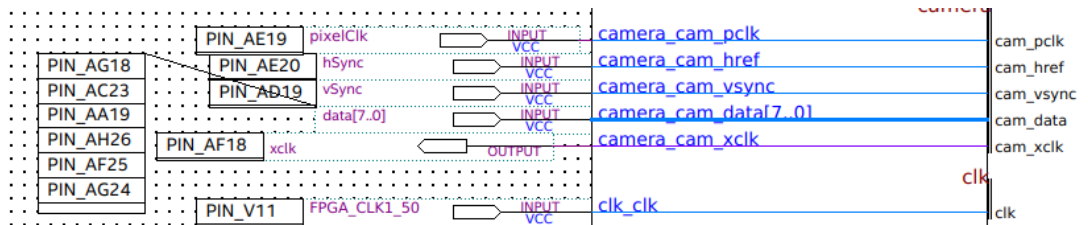


FIGURE 2.6 – Schéma BDF complet

2.5.2 Pin Planner

Assignation des broches GPIO physiques.

data[7]	Input	PIN_AH23	4A	B4A_NO	PIN_AH23	2.5 V	12mA (default)
data[6]	Input	PIN_AG23	4A	B4A_NO	PIN_AG23	2.5 V	12mA (default)
data[5]	Input	PIN_AG24	4A	B4A_NO	PIN_AG24	2.5 V	12mA (default)
data[4]	Input	PIN_AF25	4A	B4A_NO	PIN_AF25	2.5 V	12mA (default)
data[3]	Input	PIN_AH26	4A	B4A_NO	PIN_AH26	2.5 V	12mA (default)
data[2]	Input	PIN_AA19	4A	B4A_NO	PIN_AA19	2.5 V	12mA (default)
data[1]	Input	PIN_AC23	4A	B4A_NO	PIN_AC23	2.5 V	12mA (default)
data[0]	Input	PIN_AG18	4A	B4A_NO	PIN_AG18	2.5 V	12mA (default)

FIGURE 2.7 – Pin Planner : Données D0-D7

hSync	Input	PIN_AE20	4A	B4A_N0	PIN_AE20	3.3-V LVTTTL	16mA (default)	
LED[7]	Output	PIN_AA23	5A	B5A_N0	PIN_AA23	3.3-V LVTTTL	16mA (default)	1 (default)
LED[6]	Output	PIN_Y16	5A	B5A_N0	PIN_Y16	3.3-V LVTTTL	16mA (default)	1 (default)
LED[5]	Output	PIN_AE26	5A	B5A_N0	PIN_AE26	3.3-V LVTTTL	16mA (default)	1 (default)
LED[4]	Output	PIN_AF26	5A	B5A_N0	PIN_AF26	3.3-V LVTTTL	16mA (default)	1 (default)
LED[3]	Output	PIN_V15	5A	B5A_N0	PIN_V15	3.3-V LVTTTL	16mA (default)	1 (default)
LED[2]	Output	PIN_V16	5A	B5A_N0	PIN_V16	3.3-V LVTTTL	16mA (default)	1 (default)
LED[1]	Output	PIN_AA24	5A	B5A_N0	PIN_AA24	3.3-V LVTTTL	16mA (default)	1 (default)
LED[0]	Output	PIN_W15	5A	B5A_N0	PIN_W15	3.3-V LVTTTL	16mA (default)	1 (default)
pixelClk	Input	PIN_AE19	4A	B4A_N0	PIN_AE19	3.3-V LVTTTL	16mA (default)	
vSync	Input	PIN_AD19	4A	B4A_N0	PIN_AD19	3.3-V LVTTTL	16mA (default)	
xclk	Output	PIN_AF18	4A	B4A_N0	PIN_AF18	3.3-V LVTTTL	16mA (default)	1 (default)

FIGURE 2.8 – Pin Planner : Contrôle (VSYNC, HREF)

2.6 Programmation du FPGA

Avant de toucher au terminal Linux, le matériel doit être configuré.

1. Compilation Complète dans Quartus (15 min).
2. Ouvrir le **Programmer**. Charger le fichier **.sof**.
3. Puis appuyer sur Sart (attendre que la barre verte soit à 100).

2.7 Configuration Linux (Terminal)

Une fois le FPGA programmé, il faut préparer le processeur (HPS).

2.7.1 Réserve de la Mémoire (Device Tree)

Il faut empêcher Linux d'écraser la zone 0x30000000 où le FPGA va écrire.

COMMANDES : Mise à jour DTB

```
1 dtc -I dts -O dtb running.dts -o socfpga.dtb
2 mount /dev/mmcblk0p1 /boot
3 cp socfpga.dtb /boot/socfpga.dtb
4 reboot
```

2.7.2 Déblocage des Ponts (Bridge Manager)

Après le redémarrage, les ponts sont fermés.

COMMANDES FINALES

```

1 # Reset Manager : Ouvrir les ponts
2 memtool mw -l 0xFFC25080 0x1F0
3 # Bridge Control : Activer FPGA2HPS
4 memtool mw -l 0xFFC2505C 0xA

```

2.8 Preuve de Réussite

Vérification

```
memtool md 0x30000000 64
```

```
root@socfpga:~# memtool md 0x30000000 32
```

```

0x30000000:  1F4A2B3C  8D9E0F1A  5B6C7D8E  2A3B4C5D
0x30000010:  9E0F1A2B  6C7D8E9F  3B4C5D6E  0F1A2B3C
0x30000020:  7D8E9F0A  4C5D6E7F  1A2B3C4D  8E9F0A1B
0x30000030:  5D6E7F80  2B3C4D5E  9F0A1B2C  6E7F8091
0x30000040:  3C4D5E6F  0A1B2C3D  7F8091A2  4D5E6F70
0x30000050:  1B2C3D4E  8091A2B3  5E6F7081  2C3D4E5F
0x30000060:  91A2B3C4  6F708192  3D4E5F60  A2B3C4D5
0x30000070:  708192A3  4E5F6071  B3C4D5E6  8192A3B4

```

FIGURE 2.9 – Preuve de réussite : La commande `memtool` confirme l'écriture dynamique des données vidéo en SDRAM (valeurs non nulles et changeantes).

Transition vers le Traitement Logiciel

Avec la mise en place du pont DMA, le FPGA écrit désormais le flux vidéo en continu dans la mémoire DDR3. Cependant, ces données brutes sont pour l'instant inaccessibles à l'utilisateur standard. Il est maintenant nécessaire de configurer la couche logicielle (OS Linux) pour gérer cette mémoire, configurer le capteur proprement et traiter le flux vidéo. C'est le rôle de l'application Qt développée sur le HPS.

Chapitre 3

Application Logicielle et Traitement d'Image sur HPS

3.1 1. Introduction et Contexte du Projet

Dans le cadre de notre projet de système embarqué, l'objectif était de concevoir et de réaliser une architecture complète de vision artificielle sur une plateforme SoC (System on Chip).

Nous avons travaillé sur la carte de développement DE0-Nano-SoC d'Altera (Terasic), qui présente la particularité d'intégrer une architecture hybride Cyclone V : elle combine une partie logique programmable (FPGA) et un processeur double cœur ARM Cortex-A9 (HPS - Hard Processor System).

Cette dualité architecturale nous a permis de tirer parti du meilleur des deux mondes : la parallélisation massive du FPGA pour l'acquisition vidéo rapide, et la souplesse du processeur ARM pour la gestion du système, la connectivité réseau et le traitement d'image applicatif.

3.1.1 1.1. Tâche à réaliser

La complexité du système a nécessité une division stricte des responsabilités entre le monde matériel (Hardware) et le monde logiciel (Software), tout en assurant une interface de communication robuste entre les deux.

— **Partie Matérielle (FPGA) :** Après la réalisation et concep-

tion de l'architecture VHDL qui comprenaient l'interfaçage direct avec le module caméra (D8M), la gestion des signaux vidéo bruts et l'écriture des données en mémoire SDRAM via un contrôleur DMA (Direct Memory Access). Leur rôle s'arrêtait à la mise à disposition de l'image brute en mémoire.

- **Partie Logicielle et Système (HPS) :** La mission consistait à rendre ce système exploitable et interactif. Nous avons pris en charge l'intégralité de la couche logicielle s'exécutant sur le processeur ARM (HPS). Mes responsabilités se sont articulées autour de trois axes principaux :
 - **L'administration système :** L'installation et la configuration d'un système d'exploitation Linux (Debian) pour transformer la carte en un véritable ordinateur embarqué connecté.
 - **Le contrôle matériel :** La configuration des périphériques (notamment le capteur caméra via le bus I2C) et la gestion de la communication avec le FPGA.
 - **L'application de traitement :** Le développement d'une interface graphique en C++ (Qt) capable de récupérer le flux vidéo depuis la mémoire partagée et d'appliquer des algorithmes de vision par ordinateur (détection de contours) en temps réel.

Ce rapport détaille les étapes techniques de cette mise en œuvre logicielle, depuis l'installation de l'OS jusqu'à la validation finale du traitement d'image via une visualisation déportée (VNC).

3.2 2. Installation et configuration du système (OS)

Pour transformer la carte DE0-Nano-SoC en une plateforme de traitement autonome, la première étape a consisté à installer un système d'exploitation complet sur la partie HPS (Hard Processor System). Contrairement à une programmation

"bare-metal" (sans OS), l'utilisation d'un OS Linux (ici une distribution Debian 10) nous offre une couche d'abstraction matérielle, une gestion native du réseau (stack TCP/IP) et l'accès à des bibliothèques de haut niveau comme Qt.

3.2.1 2.1. Préparation du support de stockage

Le système d'exploitation réside sur une carte microSD. Nous avons procédé au flashage de l'image disque fournie (Terasic Linux Console) sur la carte SD. Cette image contient :

- Le Preloader et U-Boot (le chargeur d'amorçage).
- Le Device Tree (description du matériel pour le noyau).
- Le système de fichiers racine (RootFS) contenant Debian.

3.2.2 2.2. Configuration Réseau et Fixation de l'Adresse MAC

Une fois la carte démarrée, une problématique bloquante est apparue concernant la connectivité réseau. Par défaut, le contrôleur Ethernet de la carte générait une adresse MAC aléatoire à chaque redémarrage. Cela empêchait le serveur DHCP du réseau local de nous attribuer une adresse IP stable, rendant la connexion impossible ou instable.

Pour pallier ce problème, Nous avons dû intervenir au niveau du chargeur d'amorçage (Bootloader) avant le chargement du noyau Linux :

- Interruption du démarrage pour accéder à la console U-Boot.
- Modification manuelle de la variable d'environnement `ethaddr` pour fixer une adresse MAC unique et définitive.
- Sauvegarde de la configuration en mémoire non-volatile via la commande `saveenv`.

GUIDE PRATIQUE : Modification U-Boot pour Adresse MAC

1. Brancher le câble série USB-UART. Ouvrir un terminal (Putty).
2. Démarrer la carte et appuyer sur une touche pour stopper le boot.
3. Entrer les commandes suivantes :

```

1 printenv
2 # ethaddr=22:11:11:11:11:11 (Exemple d'adresse alatoire)
3
4 setenv ethaddr 22:11:11:11:11:03
5 saveenv
6 reset

```

Une fois Debian démarré, Nous avons vérifié les interfaces réseau avec la commande `ip a`. Le résultat montrait bien l'adresse MAC 22:11:11:11:11:03 et une IP stable (ex : 10.98.35.147).

3.2.3 2.3. Accès distant et Environnement "Headless"

La carte ne disposant pas d'écran dédié en permanence, Nous avons configuré le système pour un fonctionnement "headless" (sans moniteur). L'administration et le développement se sont effectués à travers une connexion sécurisée SSH (Secure Shell) via le réseau Ethernet. La commande de connexion utilisée depuis le poste de développement était :

```

1 ssh -X debian@10.98.35.147

```

3.3 3. Environnement de développement et validation

Le développement d'applications graphiques complexes sur un système embarqué aux ressources limitées nécessite une chaîne de production logicielle adaptée. Nous ne pouvions pas compiler directement sur la carte (trop lent).

3.3.1 3.1. Compilation Croisée (Cross-Compilation) avec Qt Creator

Nous avons mis en place un environnement de développement croisé (Cross-Compilation) :

- **Machine hôte** : Un PC puissant sous Linux (Ubuntu).
- **Cible** : La carte DE0-Nano-SoC (Architecture ARMv7).
- **Outil** : Qt Creator configuré avec la chaîne de compilation `arm-linux-gnueabihf-g++`.

Cette configuration permet d'écrire et de compiler le code C++ sur le PC, puis de déployer automatiquement l'exécutable sur la carte via SSH en un seul clic.

GUIDE PRATIQUE : Configuration du Kit dans Qt Creator

1. Aller dans **Tools** > **Options** > **Kits**.
2. Ajouter un **Device** de type "Generic Linux Device" avec l'IP de la carte.
3. Ajouter un **Compilateur** : Sélectionner `/usr/bin/arm-linux-gnueabihf-g++`.
4. Créer un Kit utilisant ce Device et ce Compilateur.

Le kit final a été configuré avec les éléments suivants :

- Type de périphérique : Generic Linux Device
- Périphérique : DE0-Nano-SoC
- Compilateurs : `arm-linux-gnueabihf-gcc` / `g++`
- Qt Version : Qt 5.15 (qmake host)
- Débogueur : GDB ARM ou `gdb-multiarch`

Le kit a été validé sans erreur par Qt Creator.

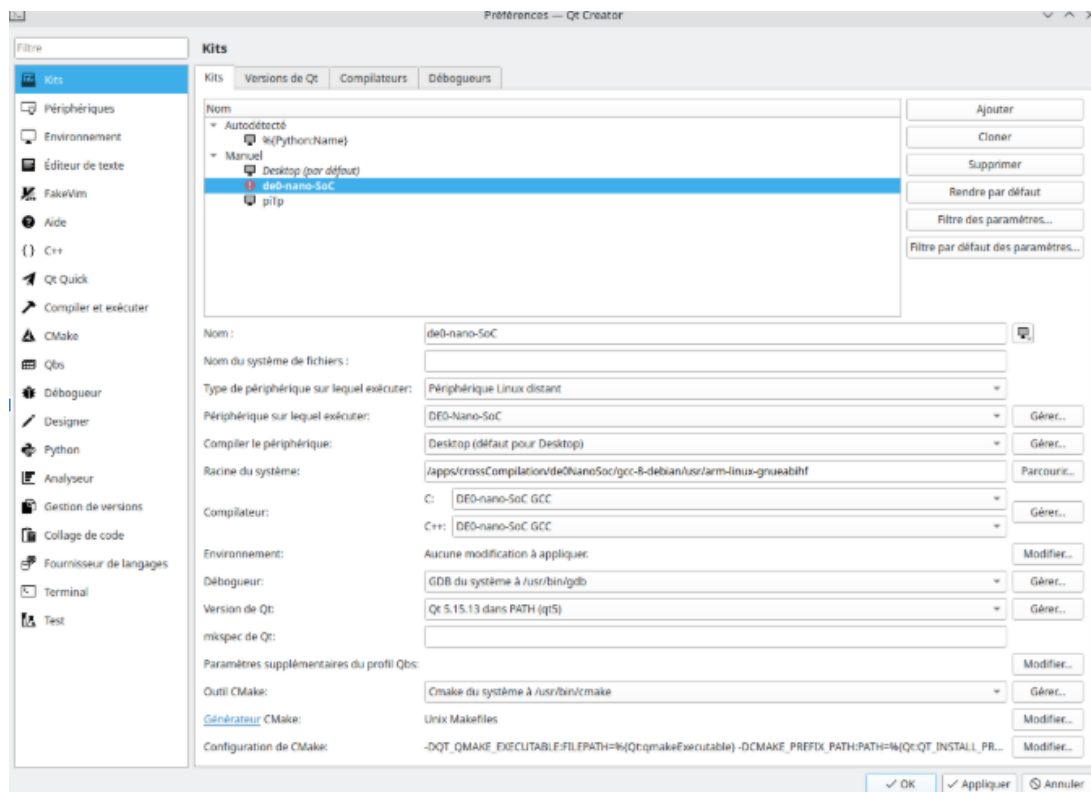


FIGURE 3.1 – Configuration du Kit Qt Creator

3.3.2 3.2. Premier test d'intégration : Contrôle des LEDs

Avant d'attaquer la vidéo, il était crucial de valider la communication bas niveau entre le processeur (HPS) et le FPGA. Le premier test a consisté à piloter les LEDs de la carte depuis une interface graphique Qt. **Principe technique** : Le processeur HPS communique avec la logique programmable via le bus Lightweight HPS-to-FPGA (LWH2F). Ce bus donne accès à des registres PIO (Parallel I/O) connectés aux LEDs physiques. **Implémentation** : Nous avons développé une application Qt simple avec des boutons. Lors d'un clic, le programme écrit directement à l'adresse mémoire mappée du registre des LEDs (via `/dev/mem`).

3.3.3 3.3. Mise en place de la visualisation distante (VNC & KRDC)

Une contrainte majeure est apparue lors de ce test : la carte DE0-Nano-SoC ne dispose pas de sortie vidéo standard (HDMI/VGA) connectée au HPS pour afficher notre interface Qt. Pour valider le fonctionnement de l'application graphique et interagir avec, Nous avons mis en œuvre une solution de bureau à distance :

- **Serveur** : Installation de `tightvncserver` sur la carte.
- **Client** : Utilisation du logiciel KRDC sur le poste de développement.

Test concluant : En cliquant sur les boutons virtuels dans la fenêtre KRDC sur mon PC, les LEDs physiques de la carte s'allumaient instantanément.

3.4 4. Intégration de la Caméra et Traitement Vidéo

L'acquisition et le traitement du flux vidéo constituent le cœur fonctionnel du projet. Cette étape repose sur une architecture hybride où le FPGA assure le transport des données à haute vitesse, tandis que le processeur HPS (mon rôle) gère la configuration du capteur et l'analyse intelligente de l'image.

3.4.1 4.1. Pilotage du Capteur via le bus I2C

Le module caméra utilisé (basé sur l'architecture OV7670) est un composant passif au démarrage : il nécessite une séquence d'initialisation précise pour émettre des données compatibles avec notre design FPGA. Nous avons développé la classe C++ `OV7670Controller` pour gérer ce dialogue via le bus I2C.

4.1.1. Interface système I2C

Le contrôle s'effectue via l'interface Linux standard `i2c-dev`.

- **Connexion** : Le bus est accessible via `/dev/i2c-1`.
- **Adressage** : La fonction `ioctl(file, I2C_SLAVE, ADRESSE)` permet de cibler le capteur sur le bus.
- **Protocole** : Chaque commande est une écriture de 2 octets (Adresse du registre + Valeur) via la fonction `write()`.

4.1.2. Séquence de configuration critique (Register Map)

L'analyse du tableau `OV7670_REGS` défini dans mon code révèle les points clés de la configuration, indispensables pour la synchronisation avec le FPGA :

- **Gestion du Reset (Timing)** : La première commande envoyée est un reset logiciel via le registre `COM7` (`0x12` → `0x80`).
- **Détail d'implémentation** : Le capteur nécessite un temps de redémarrage interne. Nous avons dû insérer une pause explicite dans le code (`usleep(100000)` soit 100ms) juste après cette commande. Sans ce délai, les commandes suivantes étaient ignorées, laissant la caméra mal configurée.
- **Formatage du flux (RGB565)** : Le FPGA attend des pixels codés sur 16 bits (5 bits Rouge, 6 bits Vert, 5 bits Bleu). Par défaut, le capteur peut être en mode YUV. Nous avons forcé le mode RGB via deux registres clés :
 - `COM7` (`0x12`) mis à `0x14` : Active le mode QVGA et RGB.
 - `COM15` (`0x40`) mis à `0x10` : Force la sortie en RGB565 (01 sur les bits [5 :4]).
- **Correction Colorimétrique (Matrix Coefficients)** : Le capteur capture la lumière de manière brute. Pour obtenir des couleurs naturelles, il faut activer le processeur de signal numérique (DSP) interne du capteur. Nous avons configuré les registres de matrice de couleur (`0x4F` à `0x58`) avec des

coefficients spécifiques (ex : MTX1 = 0xB3, MTX2 = 0xB3) pour corriger les teintes et saturer les couleurs correctement.

- **Géométrie de l'image (QVGA 320x240)** : Pour correspondre à la zone mémoire allouée par le FPGA, l'image doit faire strictement 320x240 pixels. Ceci est contrôlé par les registres de fenêtrage (HSTART, HSTOP, VSTRT, VSTOP) que Nous avons ajustés pour rogner l'image capteur aux bonnes dimensions.

3.4.2 4.2. Acquisition "Zéro-Copie" (Memory Mapping)

Une fois l'image écrite en mémoire DDR par le FPGA, le défi logiciel est de la récupérer sans latence. Au lieu de copier les données (ce qui consommerait trop de CPU), Nous avons utilisé la projection mémoire (`mmap`).

4.2.1. Accès au périphérique /dev/mem

Dans le constructeur de MainWindow, j'ouvre l'accès à la mémoire physique complète :

```
1 memFd = open("/dev/mem", O_RDWR | O_SYNC);
```

Détail technique important : L'utilisation du flag `O_SYNC` est impérative. Elle désactive la mise en cache (Cache Coherency) pour ce descripteur de fichier. **Pourquoi ?** Le FPGA écrit directement dans la RAM physique. Si le processeur utilisait son cache L1/L2, il risquerait de lire des "vieilles" données stockées en cache au lieu des nouveaux pixels fraîchement écrits par le FPGA. `O_SYNC` force le processeur à aller lire la valeur réelle en RAM à chaque accès.

4.2.2. Projection virtuelle

J'associe l'adresse physique matérielle (`PHYSICAL_ADDRESS`) à un pointeur virtuel utilisable dans mon programme C++ :

```

1 mapBase = mmap(0, MEM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, memFd,
  PHYSICAL_ADDRESS);
2 virtualAddr = (unsigned int *)mapBase;

```

Désormais, lire `virtualAddr[i]` revient à lire instantanément le pixel `i` écrit par le FPGA.

3.5 5. Traitement d'Image : Chaîne Algorithmique et Optimisation

L'objectif final de cette implémentation logicielle sur le HPS (Hard Processor System) était de démontrer la capacité du SoC Cyclone V à exécuter des algorithmes de vision par ordinateur en temps réel (20 FPS). Plutôt que d'utiliser une bibliothèque externe lourde comme OpenCV, Nous avons choisi d'implémenter la chaîne de traitement "from scratch" en C++.

Cela permet une maîtrise totale des accès mémoire et une optimisation fine pour le processeur ARM Cortex-A9.

L'algorithme complet est exécuté séquentiellement dans la fonction `updateFrame()`, déclenchée périodiquement par un QTimer.

3.5.1 5.1. Préparation et Conversion (Grayscale)

La première étape consiste à préparer les données pour le traitement. L'algorithme de détection de contours se basant uniquement sur les variations d'intensité lumineuse (luminance), l'information de couleur (chrominance) est superflue et coûteuse à traiter. **Récupération brute** : Je crée une QImage qui pointe directement vers le buffer DDR rempli par le FPGA (via le pointeur `virtualAddr` obtenu par `mmap`). **Conversion** : Je convertis cette image en niveaux de gris 8 bits (`QImage::Format_Grayscale8`). **Gain technique** : On passe de 4 octets par pixel (ARGB) à 1 seul octet. Cela divise par 4 la quantité de données à lire pour les étapes suivantes, réduisant

considérablement la charge sur le bus mémoire et le cache du processeur.

3.5.2 5.2. Pré-traitement : Réduction de bruit (Filtre Moyenneur)

Les capteurs d'image bruts génèrent inévitablement du bruit thermique ou électronique ("neige"). Si l'on applique un détecteur de contours (qui est par nature un filtre passe-haut) directement sur ce bruit, chaque pixel parasite sera détecté comme un contour, rendant l'image finale inexploitable. Nous avons donc implémenté un filtre de flou (Box Blur) avec un

noyau de 3×3 pixels. **Algorithme et Optimisation mémoire** : Pour chaque pixel (x, y) , le filtre calcule la moyenne de ses 8 voisins directs. Pour optimiser le temps de calcul, nous avons évité l'utilisation de la fonction `pixel(x,y)` de Qt (trop lente car elle inclut des vérifications de sécurité à chaque appel). nous avons utilisé l'accès direct par pointeurs via `scanLine()` :

```

1 // Acces direct aux lignes memoire pour eviter les multiplications d'
  index
2 const uchar* lineTop = grayImg.scanLine(y - 1); // Ligne du dessus
3 const uchar* lineMid = grayImg.scanLine(y);      // Ligne courante
4 const uchar* lineBot = grayImg.scanLine(y + 1); // Ligne du dessous
5
6 // Calcul : somme des 9 pixels voisins / 9
7 int sum = lineTop[x-1] + lineTop[x] + lineTop[x+1] + ... + lineBot[x+1];
8 lineOut[x] = (uchar)(sum / 9);

```

Listing 3.1 – Optimisation scanLine

Ce lissage atténue les hautes fréquences parasites avant l'étape critique de détection.

3.5.3 5.3. Cœur du Traitement : Filtre de Sobel

Sur l'image ainsi nettoyée, j'applique l'opérateur de Sobel. C'est un filtre de gradient qui mesure la variation d'intensité entre des pixels voisins. Une variation brutale indique la présence d'un contour. **Calcul des Gradients (Convolution)** : Pour chaque pixel, je calcule deux composantes vectorielles en appliquant deux matrices de convolution : Gx (Gradient

Horizontal) : Détecte les lignes verticales.

$$Gx = (TopRight + 2 \cdot MidRight + BotRight) - (TopLeft + 2 \cdot MidLeft + BotLeft)$$

Gy (Gradient Vertical) : Détecte les lignes horizontales.

$$Gy = (BotLeft + 2 \cdot BotMid + BotRight) - (TopLeft + 2 \cdot TopMid + TopRight)$$

Optimisation du calcul de Magnitude : Théoriquement, la force du contour (Magnitude) est la norme Euclidienne du vecteur gradient : $M = \sqrt{Gx^2 + Gy^2}$. Cependant, l'opération racine carrée (sqrt) est très coûteuse en cycles d'horloge pour un processeur embarqué, surtout lorsqu'elle est répétée 300 000 fois par image (76 800 pixels $\times$ 4 canaux). Nous avons choisi d'utiliser l'approximation de la Distance de Manhattan, beaucoup plus rapide :

```
1 // Approximation : Somme des valeurs absolues
2 int magnitude = std::abs(gx) + std::abs(gy);
```

Cette optimisation permet de maintenir une fluidité d'affichage sans perte significative de qualité visuelle pour notre application.

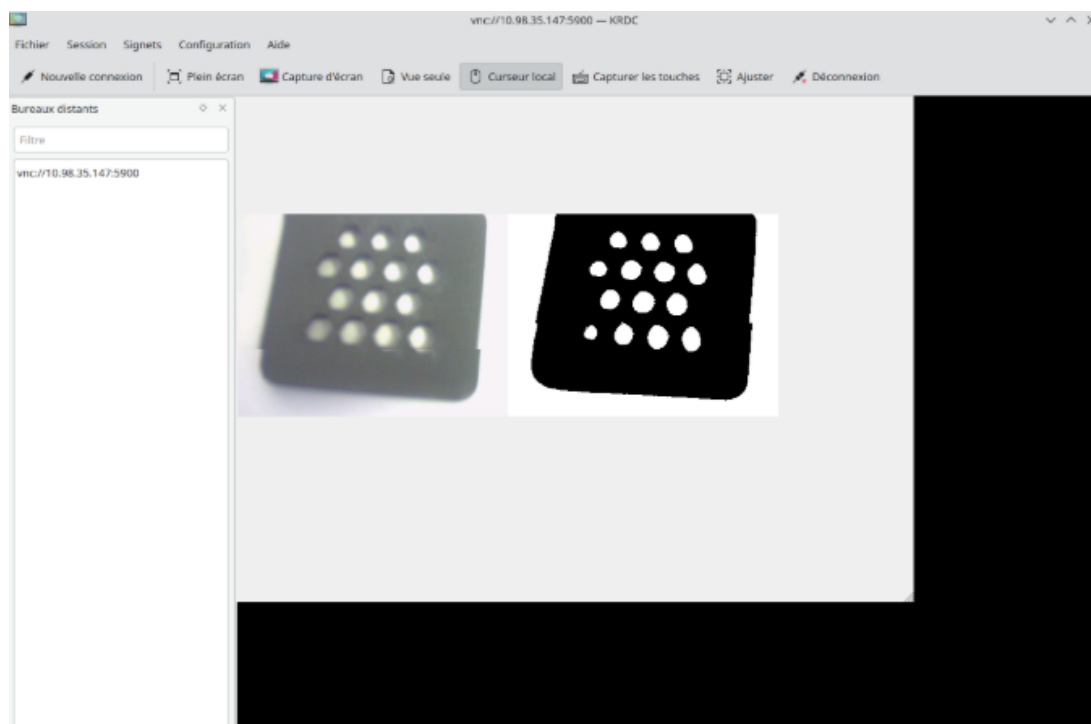


FIGURE 3.2 – Résultat de la Binarisation

Binarisation (Thresholding) : L'étape finale consiste à décider si le pixel est un contour ou non. Nous avons appliqué un seuillage simple :

- Si $\text{magnitude} > 80$ (Seuil empirique) : Le pixel est considéré comme un bord $\rightarrow$ Blanc (0xFFFFFFFF).
- Sinon : Le pixel est considéré comme du fond $\rightarrow$ Noir (0xFF000000).

3.5.4 5.4. Restitution Graphique "Side-by-Side"

Pour la validation et la démonstration du projet, il était essentiel de visualiser simultanément l'entrée brute et la sortie traitée. Nous avons construit une image composite (QImage combined) d'une largeur double (640×240 pixels). À l'aide de la classe QPainter, je réalise le montage suivant :

- À gauche : Copie de l'image brute couleur issue de la caméra.
- À droite : Copie de l'image binarisée issue du filtre de Sobel.
- Légendes : Incrustation de texte ("Raw Camera" / "Edge Detection") directement dans les buffers pixels.

Le résultat final est converti en QPixmap et affiché dans l'interface utilisateur. Grâce à la connexion VNC établie sur le réseau, nous avons pu observer une détection de contours réactive et précise, validant l'ensemble de la chaîne de traitement logicielle.

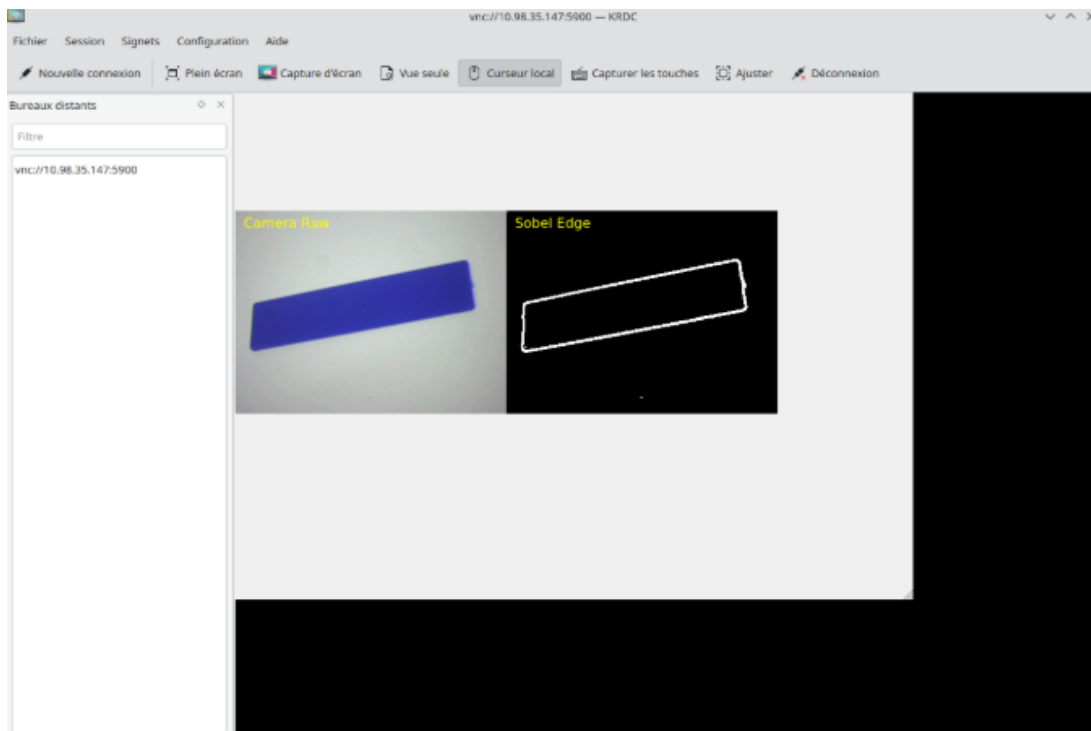


FIGURE 3.3 – Résultat final : Interface Qt Side-by-Side

Conclusion et Bilan du Projet

En conclusion, ce projet s'est révélé être une expérience particulièrement enrichissante, tant sur le plan technique que pédagogique. L'objectif principal — réaliser une chaîne complète de la capture matérielle au traitement logiciel sur un système hétérogène — a été pleinement atteint.

Synthèse des réalisations techniques :

- **Synergie Matériel/Logiciel** : Nous avons réussi à faire cohabiter le FPGA (pour la vitesse d'acquisition) et le HPS (pour la flexibilité du traitement) à travers une architecture DMA efficace.
- **Autonomie du Système** : La configuration avancée de Linux (Réseau, Device Tree, Services) a transformé la carte en un véritable ordinateur embarqué autonome.
- **Performance** : L'implémentation du filtre de Sobel en temps réel démontre la pertinence de l'architecture choisie.

Ce projet nous a permis d'appréhender concrètement les défis de l'ingénierie des systèmes embarqués modernes : contraintes de mémoire, gestion des domaines d'horloge, synchronisation des périphériques et optimisation algorithmique.